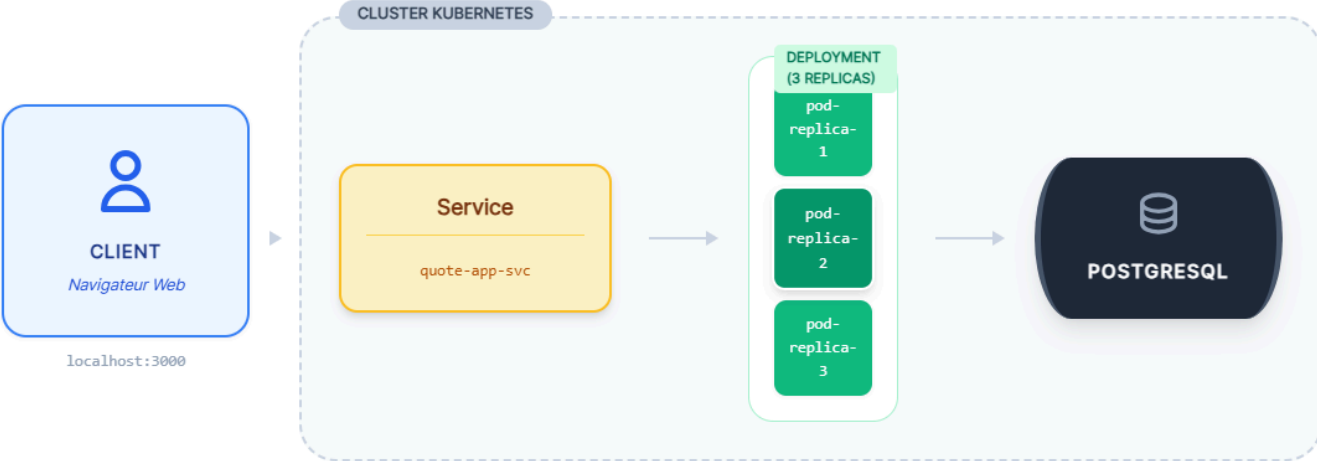


Rapport d'architecture DevOpsCody - KUB TP 85

Step 1 2 3

Architecture de Production : Flux de Données Horizontal

Visualisation du parcours d'une requête dans le cluster



Analyse du Diagramme :

1. L'isolation : Elle se produit au niveau du Pod (chaque instance a son propre espace réseau et système de fichiers) et du Cluster (isolé de l'infrastructure physique).
2. Redémarrage automatique : Si un Pod échoue, le Deployment détecte que l'état actuel ne correspond pas à l'état désiré et recrée un nouveau Pod.
3. Ce que Kubernetes ne gère pas : Le matériel physique du serveur (le CPU/RAM physique), la connectivité réseau Internet avant d'entrer dans le cluster, et le code source de l'application lui-même.

Tableau Comparatif

Caractéristique	Conteneurs (ex: Docker/Kubernetes)	Machines Virtuelles (ex: VMware/KVM)
Partage du Noyau	Partagent le noyau du système d'exploitation hôte.	Chaque VM possède son propre noyau (Guest OS) complet.
Temps de Démarrage	Quasi instantané (quelques secondes).	Lent (plusieurs minutes pour booter l'OS).
Surcharge (Overhead)	Très faible : utilise les ressources de l'hôte directement.	Élevée : nécessite des ressources pour chaque OS invité.

Caractéristique	Conteneurs (ex: Docker/Kubernetes)	Machines Virtuelles (ex: VMware/KVM)
Isolation de Sécurité	Isolation au niveau processus (moins hermétique).	Isolation matérielle via l'hyperviseur (très forte).
Complexité Opérationnelle	Élevée pour l'orchestration, mais facilite le CI/CD.	Modérée pour la gestion unitaire, lourde à l'échelle.

On privilégiera la VM dans les scénarios suivants :

1. **Isolation stricte (Multi-tenancy) :** Lorsque vous exécutez du code provenant de sources non fiables sur le même matériel physique, l'isolation au niveau du noyau de la VM est plus sûre.
2. **Besoin d'un OS spécifique :** Si l'application nécessite un noyau Windows alors que l'hôte est Linux (ou vice-versa).
3. **Applications monolithiques :** Les applications anciennes qui ne supportent pas d'être découpées en micro-services ou qui nécessitent un état système persistant très complexe.

La combinaison est en réalité le standard de l'industrie (notamment dans le Cloud) :

- **Infrastructure Cloud (IaaS) :** Les fournisseurs comme AWS ou GCP fournissent des VM (instances) sur lesquelles nous installons des clusters Kubernetes. Cela permet de bénéficier de l'isolation matérielle des VM pour la sécurité des nœuds, tout en profitant de l'agilité et de la densité des conteneurs pour les applications.
- **Environnements Hybrides :** Utiliser des VM pour les bases de données critiques (pour la stabilité et la performance disque brute) et des conteneurs pour les serveurs web et API (pour la scalabilité rapide).

Step 4 : Ce qui change lors du passage à l'échelle :

- **Nombre de Pods :** Le cluster exécute désormais trois instances indépendantes de l'application.
- **Disponibilité :** La redondance est accrue. Si un Pod tombe en panne, les deux autres continuent de servir le trafic.
- **Répartition de la charge :** Le Service Kubernetes agit comme un équilibreur de charge (Load Balancer) et distribue les requêtes entrantes entre les trois réplicas.
- **Identifiants réseau :** Chaque nouveau Pod possède sa propre adresse IP interne et son propre nom unique.

Ce qui ne change pas :

- **Point d'entrée unique :** L'adresse IP du Service et son nom DNS restent identiques. L'utilisateur (ou le port-forward) ne voit pas qu'il y a plusieurs Pods derrière.
- **Configuration :** L'image du conteneur, les variables d'environnement et les ressources allouées restent les mêmes pour chaque réplica.
- **État de la base de données :** Tous les Pods se connectent à la même instance de base de données PostgreSQL. L'application doit être "stateless" (sans état) pour que cela fonctionne correctement.

Step 5 : Simuler une failure

On peut voir que :

- Qui a recréé le Pod ? C'est le **ReplicaSet** controller (piloté par le Deployment).
- Pourquoi ? Kubernetes compare constamment l'état désiré (3 réplicas) à l'état actuel (2 réplicas après la suppression). Dès qu'un écart est détecté, il crée un nouveau Pod pour rétablir l'équilibre.
- Et si le nœud complet échouait ? Si un serveur (nœud) tombe, Kubernetes détecte que les Pods ne répondent plus. Il attend un délai de grâce, puis replanifie automatiquement ces Pods sur d'autres nœuds sains du cluster pour garantir la continuité du service.

```
quote-app-69746878f-v95rh 1/1 Running 0 2m39s
nath@PINF-NCARQ25:/mnt/c/Users/carquein/Documents/_Ecole/DevOps/res507-2025-2026-work/labs/40-containerized-node-app$ kubectl apply -f deployment.yaml
kubectl get pods
deployment.apps/quote-app unchanged
NAME                                READY   STATUS    RESTARTS   AGE
nginx-test-586bbf5c4c-d8785        1/1     Running   0           4h5m
quote-app-64bf9d6589-t2bcc         0/1     ErrImageNeverPull 0           8s
quote-app-69746878f-jc5mw         1/1     Running   0           3m1s
quote-app-69746878f-kglps         1/1     Running   0           2m53s
quote-app-69746878f-v95rh         1/1     Running   0           2m46s
nath@PINF-NCARQ25:/mnt/c/Users/carquein/Documents/_Ecole/DevOps/res507-2025-2026-work/labs/40-containerized-node-app$ |
```

Step 6 : Ressource limits

- Requests (Requêtes) : Le minimum garanti. Kubernetes utilise cette valeur pour décider sur quel nœud placer le Pod.
- Limits (Limites) : Le plafond maximum. Si le Pod dépasse cette limite CPU, il est bridé ; s'il dépasse la limite mémoire, il est tué (OOMKilled).
- Importance en multi-tenant : Cela empêche un utilisateur ou un service de monopoliser toutes les ressources du serveur, garantissant que les autres applications ont toujours accès à leur part de CPU/RAM.

Step 7 : Liveness readiness

- **Différence entre Readiness et Liveness :**
 - Liveness : Vérifie si l'application est "vivante". Si elle échoue, K8s redémarre le Pod.
 - Readiness : Vérifie si l'application est prête à recevoir du trafic. Si elle échoue, K8s retire le Pod du Service (pas de trafic envoyé), mais ne le tue pas.
- Importance en production : Cela permet d'éviter d'envoyer des clients vers un Pod qui est encore en train de démarrer ou qui est temporairement incapable de traiter des requêtes (ex: connexion DB perdue).

Connect Kubernetes to virtualization

Sous k3s : k3s tourne généralement sur un OS Linux (souvent lui-même dans une VM comme Multipass ou une instance cloud).

Remplacement ? Non, Kubernetes orchestre les conteneurs qui tournent souvent sur des VMs fournies par des clouds.

Contextes d'utilisation :

- **Cloud Data Center :** Serveurs physiques -> Hyperviseur -> VMs (Nœuds) -> Kubernetes.
- **Automobile :** Matériel embarqué -> Hyperviseur temps réel -> Nœuds légers (k3s) -> Applications de conduite.
- **Banque :** Cloud privé -> VMs isolées par département -> Clusters K8s dédiés par zone de sécurité.

Step 8 : Design d'Architecture de Production

Composants :

- **Multi-nœuds :** Au moins 3 nœuds pour la haute disponibilité.
- **Persistence :** Volumes persistants (PVC) sur stockage réseau (EBS, Longhorn) ou base de données gérée (RDS).
- **Backup :** Velero pour les ressources K8s et snapshots réguliers des volumes.
- **Monitoring/Logging :** Prometheus/Grafana pour les métriques, ELK Stack pour les logs.
- **CI/CD :** Pipeline automatisé (GitLab CI/GitHub Actions) poussant vers un registre privé.

Répartition :

- **Dans Kubernetes :** Web apps, APIs, micro-services, outils de monitoring.
- **Dans des VMs :** La base de données (si non gérée) pour de meilleures performances disques et une isolation accrue.
- **Hors cluster :** Load Balancer externe, Cloud Storage (S3), Managed Database

Step 9 : Required break and analysis

ors de la mise en place d'une image invalide (`quote-app:broken`), `kubect1 get events` montre des erreurs `ImagePullBackoff`. Cela confirme que Kubernetes ne peut pas démarrer l'application si l'image est introuvable, maintenant l'ancien Pod jusqu'à ce que le nouveau soit prêt (Rolling Update).

Step 10 : Required extension: secret-based configuration

- Les credentials ne sont plus en clair dans le code ou le fichier YAML. Cela permet une gestion centralisée et plus sécurisée.
 - **Chiffrement :** Par défaut, les Secrets sont simplement encodés en **Base64** (pas chiffrés). Le chiffrement doit être activé au repos sur le serveur API ou via un KMS externe.
-

Session 3 : Rollouts contrôlés et rollback sécurisé

Rollout v1 → v2

Changement effectué : Mise à jour du titre de la page (`QuoteBoard` → `QuoteBoard v2`) dans `app/views/index.hbs`.

Construction des images :

```
# v1 : tag de l'image existante
docker tag quote-app:local quote-app:v1

# Modification du titre dans index.hbs

# v2 : build avec le changement
docker build -t quote-app:v2 -f docker/Dockerfile .
```

Mise à jour du Deployment :

```
kubectl set image deployment/quote-app quote-app=quote-app:v2 -n quote-lab
```

Observation du rollout :

```
kubectl rollout status deployment quote-app -n quote-lab

waiting for deployment "quote-app" rollout to finish: 1 out of 3 new replicas have been updated...
waiting for deployment "quote-app" rollout to finish: 2 out of 3 new replicas have been updated...
waiting for deployment "quote-app" rollout to finish: 1 old replicas are pending termination...
deployment "quote-app" successfully rolled out
```

Historique :

REVISION	CHANGE-CAUSE
1	<none>
2	kubectl set image deployment/quote-app quote-app=quote-app:v2

Ce qui a changé pendant le rollout :

- Kubernetes a créé 1 nouveau Pod v2 (maxSurge: 1), l'a attendu en Ready, puis a supprimé 1 ancien Pod v1.
- Ce processus s'est répété jusqu'à remplacement complet des 3 réplicas.
- À aucun moment l'application n'était indisponible (grâce à maxUnavailable: 0).

Ce qui n'a pas changé :

- L'IP et le nom DNS du Service sont restés identiques.
- La stratégie de déploiement et le nombre de réplicas (3) n'ont pas changé.

- Kubernetes a décidé de créer/supprimer les Pods en vérifiant la readinessProbe : un Pod v2 est considéré prêt uniquement quand `/health` répond 200.

Rollout cassé (contrôlé)

Failure introduite : Image inexistante `quote-app:broken` avec `imagePullPolicy: Never`.

```
kubectl set image deployment/quote-app quote-app=quote-app:broken -n quote-lab
```

Observation du failure :

```
kubectl get pods -n quote-lab
```

NAME	READY	STATUS	RESTARTS	AGE
quote-app-6b8bc997cf-6p94z	0/1	ErrImageNeverPull	0	20s
quote-app-c8f75b854-4qmms	1/1	Running	0	5m51s
quote-app-c8f75b854-d2hgj	1/1	Running	0	5m58s
quote-app-c8f75b854-thmvq	1/1	Running	0	5m44s

Events capturés :

```
Warning ErrImageNeverPull pod/quote-app-6b8bc997cf-6p94z
  Container image "quote-app:broken" is not present with pull policy of Never
Warning Failed pod/quote-app-6b8bc997cf-6p94z
  Error: ErrImageNeverPull
```

Analyse :

- Ce qui a échoué en premier : le nouveau Pod n'a pas pu démarrer (`ErrImageNeverPull`).
- Le signal le plus rapide : `kubectl get pods` — le statut `ErrImageNeverPull` est visible immédiatement.
- Grâce à `maxUnavailable: 0`, les 3 anciens Pods v2 sont restés en service pendant toute la durée du failure.
- En production : vérifier l'existence du tag dans le registre, les droits d'accès (imagePullSecret), et les logs d'événements.

Rollback sécurisé

```
kubectl rollout undo deployment quote-app -n quote-lab
```

Vérification après rollback :

```
kubectl get pods -n quote-lab
```

NAME	READY	STATUS	RESTARTS	AGE
quote-app-c8f75b854-4qmms	1/1	Running	0	6m31s
quote-app-c8f75b854-d2hgj	1/1	Running	0	6m38s
quote-app-c8f75b854-thmvq	1/1	Running	0	6m24s

Historique final :

REVISION	CHANGE-CAUSE
1	<none>
3	kubectl set image ... quote-app=quote-app:broken
4	kubectl set image ... quote-app=quote-app:v2 ← rollback vers v2

Ce que le rollback a changé :

- L'image du Deployment est revenue à `quote-app:v2`.
- Les Pods "broken" ont été supprimés et remplacés par des Pods v2 sains.

Ce que le rollback n'a pas changé :

- Les Secrets, ConfigMaps, et le Service sont restés intacts.
- L'historique du Deployment est conservé (la révision cassée reste visible).
- Les données en base de données ne sont pas affectées.

Extension choisie : Option A — Stratégie de rollout explicite

Le Deployment contient maintenant :

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

maxSurge: 1 — Kubernetes peut créer 1 Pod supplémentaire au-delà des 3 désirés pendant le rollout. Cela garantit qu'un nouveau Pod est prêt avant qu'un ancien soit supprimé.

maxUnavailable: 0 — Aucun Pod ne peut être supprimé avant qu'un remplaçant soit prêt. Cela garantit 0% de perte de disponibilité pendant la mise à jour.

Pourquoi choisir 0 pour maxUnavailable ? En production, on ne veut jamais réduire la capacité disponible pendant un déploiement. Avec `maxUnavailable: 0`, l'application reste à pleine capacité tout au long du rollout, au prix d'un léger surcoût temporaire en ressources (le Pod supplémentaire).

Tests end-to-end

- **Ce que testeraient des E2E tests :** Chargement de la page d'accueil, soumission d'une nouvelle quote via le formulaire, vérification que la quote apparaît dans la liste, validation du endpoint `/health`, et test que `/api/quotes` retourne bien du JSON.
- **Où les exécuter :** Localement lors du développement pour un feedback rapide, et en CI (GitHub Actions) après chaque push avant le déploiement en production.
- **Principal coût/risque :** Les tests E2E nécessitent une base de données active, ce qui complexifie l'environnement CI. Ils sont lents (secondes à minutes) par rapport aux tests unitaires (millisecondes), et peuvent être fragiles si l'interface change souvent.